

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-10^6 \leq \text{nums}[i] \leq 10^6$

Program :

```
class Solution {  
public:  
    vector<int> runningSum(vector<int>& nums) {  
        int sum = 0;  
        for (int i = 0; i < nums.size(); i++) {  
            sum += nums[i];  
        }  
    }  
};
```

```
        nums[i] = sum;
    }

    return nums;
}
};
```

Output :

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =
[1,2,3,4]
```

Output

```
[1,3,6,10]
```

Expected

```
[1,3,6,10]
```